

实验 8：运动规划

概述

在本实验中，我们将实现用于路径规划的 A* 算法。我们将提供一个网格地图，其中一些网格点会被标记为障碍物。我们的任务是使用 A* 算法，在不与任何障碍物碰撞的情况下，找到从起始网格点到目标网格点的最优路径。你可以使用欧几里得距离作为启发式函数（或其他符合要求的启发式函数）。不允许机器人在对角线方向上移动（即只使用 4 连通图）。

作业提交

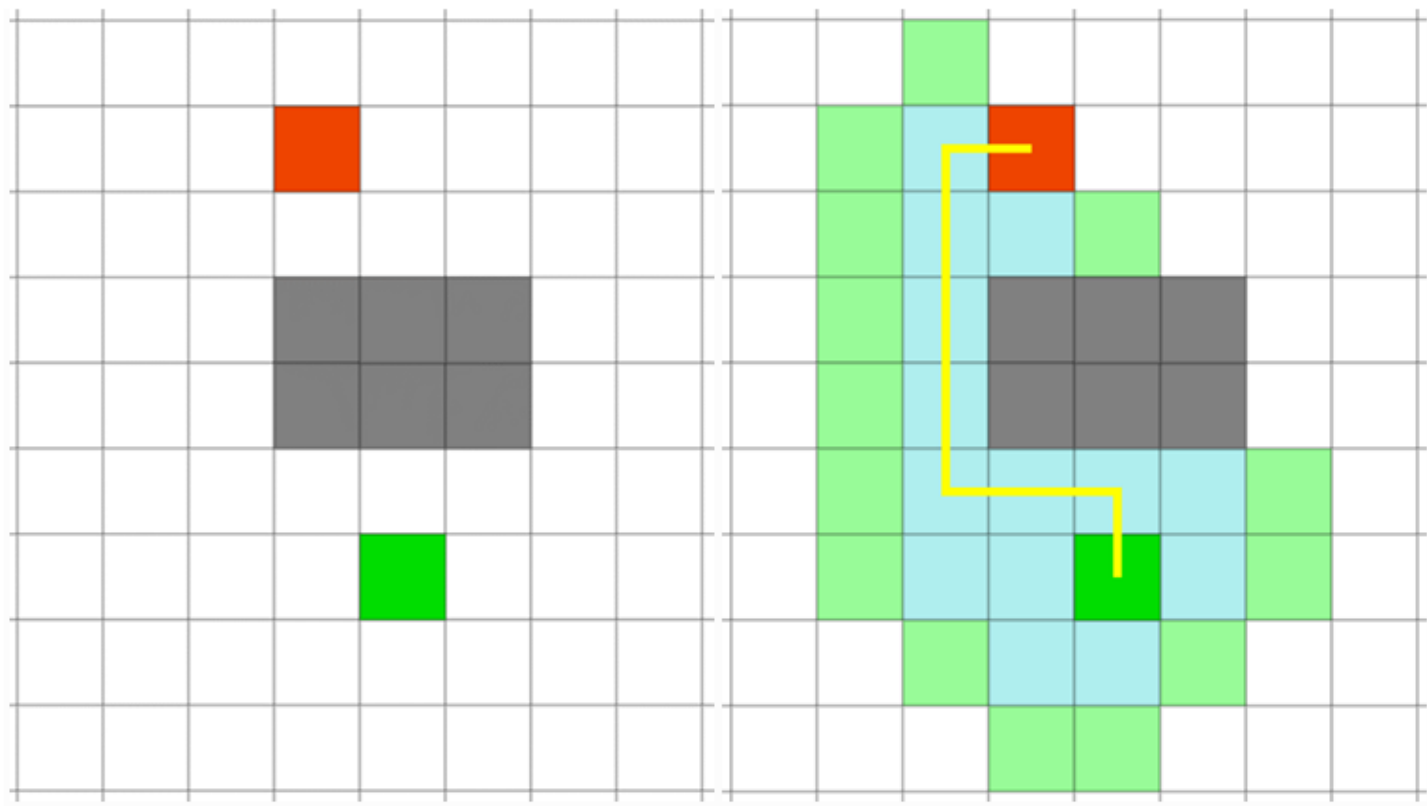
- 需提交的文件：
 - `motion_planning.py`
 - 以PDF文件格式提交实验报告，包含机器人运动轨迹的截图，并简要叙述实现过程
- 评分标准：
 - 实现 A* 算法。我们会使用一些测试案例来评估。将根据通过的测试用例数量给予部分分数。

对规划路径的要求

1. 返回的路径应该是一个Python列表，列表中的每个元素应该是一个二元组。（我们很快会讨论这个数据类型。）
2. 路径不应与任何障碍物发生碰撞。
3. 路径应包含目标网格，但不包含起始网格。
 - 例如，从(0, 0)到(2, 2)的路径应该是[(1, 0), (1, 1), (2, 1), (2, 2)]
 - 这是基于网格的运动规划算法中的一个惯例，这样在遍历列表时，它可以直接引导机器人到达目标。
4. 路径上的所有网格点之间都应该具有 4 连通性。
 - 换句话说，在每一步中，从一个网格到下一个网格，机器人只能在四个方向之一（[1, 0]、[-1, 0]、[0, 1]或[0, -1]）中移动。不允许对角线移动。
 - 起始网格和路径中的第一个网格也应该具有4连通性。

网格地图示例

下面展示了一个网格地图和其对应的 A* 算法路径的示例。



编程提示

1. Python 列表的一些用法回顾

- `path = list()` 或 `path = []` 创建一个空列表。
- `path.append()` 会在列表末尾添加一个新元素。
- `path.pop()` 移除（并返回）指定位置的元素。
- `path.sort()` 默认将列表按升序排序。
- `path.reverse()` 反转元素的排序顺序。

2. Python 中的元组 (Tuple) 数据类型

- 在 Python 中, `[0, 0]` 是一个包含 2 个元素的列表, 而 `(0, 0)` 是一个包含 2 个元素的元组 (称为二元组)。
- `[[0, 1], [1, 1], [1, 2]]` 是一个列表, 其元素也是列表, 而 `[(0, 1), (1, 1), (1, 2)]` 是一个列表, 其元素是元组。
- **路径应该是一个包含多个元素的列表, 每个元素都是一个二元组。**

3. Python 中的字典 (Dict) 数据类型

- `d = dict()` 或 `d = {}` 创建一个空字典。
- 在这个实验中我们必须使用元组, 因为元组是可哈希的, 因此可以在字典中使用。
- 如果 `start` 是列表, 则操作 `d[start] = 0` 无效; 如果 `start` 是元组, 则操作才有效。

4. 列表、元组和字典的比较

- 列表是有序且可变的。允许重复成员。
- 元组是有序且不可更改的。允许重复成员。
- 字典是无序的、可变的、有索引的集合。没有重复的成员。